



SCRIPT PRESENTASI - SLIDE 21-30

Activity Diagrams (lanjutan) & Sequence Diagrams

SLIDE 21: AD-14 - ASSIGN DRIVER (1.5 menit)

Narasi:

"Ini adalah **AD-14 Assign Driver** - proses penugasan driver untuk pengiriman.

[Point ke swimlane]

Ada 2 swimlane: **User** dan **Sistem**.

[Trace flow dari START]

User buka detail pesanan, klik tombol **Assign Driver**.

[Point ke proses sistem]

Sistem:

- **Load daftar driver aktif**
- **Cek status kesibukan** setiap driver
- Tampilkan modal dengan daftar driver

[Point ke note]

Perhatikan note: Driver yang **sedang mengantar** ditampilkan dengan badge "Mengantar" tapi **tidak bisa dipilih**.

[Point ke flow setelah pilih driver]

User pilih driver yang tersedia. Sistem kemudian:

- **Assign driver ke pesanan**
- **Auto update status** → **DIKIRIM**
- **Buat timeline track**

[Kesimpulan]

Satu aksi = dua proses (assign + update status). Ini memastikan data tracking **konsisten** dan driver tidak bisa double-booking."

SLIDE 22: AD-16 - CETAK DOKUMEN (INVOICE/NOTA) (2 menit)

Narasi:

"Ini adalah **AD-16 Generate Invoice / Nota** yang menggambarkan proses pembuatan dokumen untuk pesanan.

[Point ke swimlane]

Ada 2 swimlane: **Admin/Operator** dan **Sistem**.

[Trace flow dari START]

Proses dimulai dengan user membuka detail pesanan, lalu klik tombol **Cetak Invoice/Nota**.

[Point ke proses sistem]

Sistem merespons dengan:

- **Navigasi ke halaman dokumen**
- **Fetch data order** dari database

[Point ke catatan perbedaan Invoice vs Nota]

Perhatikan ada **catatan** yang menjelaskan perbedaan:

- **Invoice** = pre-payment (sebelum bayar)
- **Nota** = post-payment (setelah lunas)
- Keduanya di-render on-demand dari data Order

[Point ke decision - order dibayar?]

Ada **decision point**: apakah order sudah dibayar?

- Jika **belum lunas**: tampilkan error (tidak bisa cetak nota)
- Jika **sudah lunas**: lanjut proses

Sistem kemudian:

- **Load company profile** untuk header dokumen
- **Cek status pembayaran**

[Point ke decision kedua - sudah lunas?]

Jika sudah lunas, user bisa pilih: **Invoice saja** atau **Invoice dan Nota**.

Sistem kemudian:

- **Generate nomor dokumen** unik
- **Render dokumen** dengan template yang sesuai

[Point ke output dan aksi user]

User dapat:

- **Lihat dokumen** preview
- **Cetak (Print)** ke printer
- **Share WhatsApp** langsung ke pangkalan
- **Copy Link** untuk dibagikan

Diagram ini menunjukkan **fleksibilitas output** dokumen untuk kebutuhan bisnis yang berbeda."

SLIDE 23: AD-07 - CATAT PENJUALAN (PANGKALAN) (2.5 menit)

Narasi:

"Ini adalah **AD-07 Catat Penjualan** yang khusus untuk aktor **Pangkalan** - menunjukkan proses pencatatan penjualan ke konsumen akhir.

[Point ke swimlane]

Ada 2 swimlane: **Pangkalan** (bukan Admin/Operator!) dan **Sistem**. Ini menunjukkan bahwa diagram ini untuk user dengan role Pangkalan.

[Trace flow dari START]

Proses dimulai dengan pangkalan membuka halaman Penjualan, lalu klik **Catat Penjualan**.

[Point ke proses sistem - load data]

Sistem merespons dengan:

- **Load daftar konsumen** yang sudah terdaftar di pangkalan ini
- **Load stok LPG** yang tersedia di pangkalan
- **Load harga LPG** yang berlaku

[Point ke catatan multi-tenant]

Perhatikan: data yang di-load **hanya milik pangkalan yang login**. Ini adalah implementasi **multi-tenant** - pangkalan A tidak bisa lihat data pangkalan B.

[Point ke input user]

Pangkalan kemudian:

- **Pilih konsumen** dari daftar (atau input baru untuk walk-in)
- **Masukkan jenis LPG dan jumlah** yang dibeli
- **Pilih harga** (jika ada variasi)
- **Masukkan jumlah** pembayaran

[Point ke validasi stok]

Sistem melakukan **validasi stok** - apakah stok cukup?

- Jika **tidak cukup**: tampilkan error, minta order ke agen
- Jika **cukup**: lanjut proses penyimpanan

[Point ke auto-deduct stok]

Jika stok mencukupi, pangkalan **konfirmasi penjualan**. Sistem kemudian:

- **Create consumer_order** transaksi
- **Update stok pangkalan** - otomatis **KURANGI** sesuai quantity yang dijual
- **Buat stock_movement** dengan status 'KELUAR'
- **Buat pangkalan_stock_movement** untuk audit trail

[Point ke hasil akhir]

Terakhir, tampilkan pesan sukses dan refresh dashboard.

[Kesimpulan]

Ini adalah kebalikan dari **auto-sync** saat terima dari agen. Di sini stok **otomatis berkurang** saat jual ke konsumen. Pangkalan tidak perlu update stok manual - semua **real-time dan accurate**."

SLIDE 24: AD-17 - VOICE ORDER AI (3 menit)

Narasi:

"Ini adalah **AD-17 Voice Order AI** - fitur **INOVASI** yang membedakan SIM4LON dari sistem distribusi lainnya.

[Point ke swimlane]

Ada 2 swimlane: **Admin/Operator** dan **Sistem**.

[Trace flow dari START]

Proses dimulai dengan user membuka halaman pesanan, lalu klik **icon microphone** untuk Voice Order.

[Point ke request izin]

Sistem meminta **izin akses microphone** dari browser. Jika user menolak, proses berhenti dengan error.

[Point ke recording]

Jika diizinkan, sistem masuk ke mode **recording**. Ada note: 'Teks sebelum 2 detik = dianggap belum fix'.

Sistem **record audio** dari user. User berbicara, misalnya: *'Lima puluh tabung tiga kilo ke Pangkalan Sumber Rezeki'*

[Point ke speech-to-text]

Sistem kemudian:

- **Konversi ke teks** menggunakan Web Speech API browser
- **Tampilkan transkrip** untuk konfirmasi

[Point ke AI processing - HIGHLIGHT]

Inilah bagian **AI**. Sistem mengirim transkrip ke **Google Gemini AI** untuk parsing, parsing itu apa?jadi parsing itu mengekstrak informasi2 penting memecah menjadi beberapa bagian seperti jumlah, jenis LPG, dan nama pangkalan dari kalimat yang diucapkan user.

Google Gemini AI ini menerima prompt dengan format yang sudah ditentukan, kemudian mengekstrak:

- *Quantity: 50*

- *LPG Type*: 3kg
- *Pangkalan Name*: 'Pangkalan Reon'
- *Confidence Level*: 95%

[Point ke fuzzy matching]

Sistem melakukan **query database** untuk matching pangkalan. 'Pangkalan Reon' di-match ke 'Pangkalan Reon' menggunakan fuzzy search. fuzzy search itu apa? fuzzy search itu mencari data yang paling mirip dengan data yang diinput user.

Lalu **mengecek LPG yg valid**.

[Point ke konfirmasi user]

Sistem menampilkan **dialog konfirmasi** dengan hasil parsing. User bisa:

- **Konfirmasi** jika benar
- **Reset** jika ingin ulang dari awal
- Atau **edit manual** jika ada yang salah

[Point ke auto-fill form]

Jika user konfirmasi, sistem **auto-fill form** pesanan dengan data yang sudah di-parse. User tinggal review dan klik **Simpan**.

Proses selanjutnya sama dengan create order biasa - masuk ke flow AD-02 Buat Pesanan.

[Point ke hasil akhir]

Jika form sudah di-fill dengan benar, user simpan dan pesanan tercreate.

[Kesimpulan - HIGHLIGHT IMPACT]

Fitur ini **mengubah proses 5 menit menjadi 30 detik**. Sangat berguna saat kondisi lapangan sibuk - operator tidak perlu ketik, cukup bicara. Ini adalah contoh penerapan **AI dalam konteks bisnis nyata** yang memberikan **efisiensi 90%**."

SLIDE 25: SD-01 - LOGIN (SEQUENCE DIAGRAM) (2.5 menit)

Narasi:

"Sekarang kita masuk ke **Sequence Diagram** untuk menjelaskan **technical implementation** dari sistem sim4lon.

Sequence Diagram pertama adalah **SD-01 Login**.

[Point ke lifelines - participants]

Ada **5 participants**:

- **User** (aktor) yang melakukan login
- **LoginPage** (boundary) - halaman frontend

- **AuthService** (control) - service backend untuk autentikasi
- **users** (database) - tabel user
- **activity_logs** (database) - tabel log aktivitas

[Trace flow menampilkan halaman]

Proses dimulai saat User buka halaman Login. LoginPage merespons dengan tampilkan form login.

[Trace flow proses login]

User input email dan password, lalu klik tombol Login.

LoginPage memanggil `AuthService.login(email, password)`.

AuthService melakukan **query database**:

```
SELECT * FROM users WHERE email = ?
```

[Point ke ALT fragment - user tidak ditemukan]

Ada **ALT fragment** yang menangani beberapa kondisi:

- Jika **user tidak ditemukan** → throw `UnauthorizedException` → Tampilkan error 'Email atau password salah'
- Jika **user ditemukan**, lanjut ke validasi password dengan `bcrypt.compare(password, hash)`

[Point ke nested ALT - password]

- Jika **password tidak cocok** → throw `UnauthorizedException`
- Jika **password cocok**, cek status akun:
 - **Akun tidak aktif** → throw `UnauthorizedException` 'Akun tidak aktif'
 - **Akun aktif** → lanjut proses login sukses

[Point ke proses login sukses]

Untuk login sukses, AuthService:

1. **Generate session_id baru** - `generateSessionId()`
2. **UPDATE user** di database dengan session_id baru
3. **Generate JWT token** - `generateJwtToken(payload)`
4. **INSERT activity_log** dengan type 'user_login'

[Point ke note - Single Session]

Perhatikan ada **note** yang menjelaskan **Single-Session Login**: session_id baru akan invalidasi session lama. Artinya user hanya bisa login di satu device.

[Point ke return dan redirect]

AuthService return access_token ke LoginPage. LoginPage simpan token di localStorage, lalu redirect ke Dashboard.

Diagram ini menunjukkan **security berlapis** dan **audit trail** untuk setiap login."

SLIDE 26: SD-03 - CREATE ORDER (SEQUENCE DIAGRAM) (2.5 menit)

Narasi:

"Sequence Diagram kedua adalah **SD-03 Create Order** - proses pembuatan pesanan baru.

[Point ke lifelines]

Ada **7 participants**:

- **Admin** (aktor)
- **OrderPage** (boundary)
- **OrderService** (control)
- **pangkalans, orders, order_items, timeline_tracks** (databases) di akhiran table nya pake s ini karena table nya berisi data lebih dari satu. dan prisma orm default nya pake s di akhiran table.

[Trace flow menampilkan form]

Admin klik button "Buat Pesanan" untuk buka halaman Buat Pesanan. OrderPage memanggil `OrderService.getPangkalanList()`.

OrderService query database:

```
SELECT * FROM pangkalans WHERE is_active = true
```

Return daftar pangkalan aktif, lalu tampilkan form pesanan.

[Trace flow input data]

Admin pilih pangkalan, pilih jenis LPG dan quantity, lalu klik Simpan.

OrderPage memanggil `OrderService.createOrder(orderDto)`. dto itu singkatnya data transfer object yang berisi data yang akan di transfer ke server.

[Point ke proses internal OrderService]

OrderService melakukan beberapa proses:

1. **validateInput(orderDto)** - validasi data input
2. **Generate code** - query `SELECT MAX(code) FROM orders`, lalu `generateNewCode()` → hasil misalnya 'ORD-0100'

[Point ke LOOP fragment]

Ada **LOOP fragment** 'untuk setiap item': jadi misal admin pilih 2 item maka akan dihitung 2 kali. terus dihitung subtotal, check apakah kena pajak, dan hitung PPN jika NON_SUBSIDI.

- `calculateSubtotal(qty, price)` - hitung subtotal

- `checkTaxable(lpg_category)` - cek apakah kena pajak
- `calculateTax(12%)` - hitung PPN jika NON_SUBSIDI

3. `calculateTotalAmount()` - hitung total keseluruhan

[Point ke INSERT ke database]

Kemudian:

- **INSERT INTO orders** dengan status DRAFT → dapat orderId
- **INSERT INTO order_items** untuk setiap item akan di insert ke database
- **INSERT INTO timeline_tracks** dengan deskripsi 'Pesanan dibuat' → untuk menambahkan log timeline

[Point ke note - PPN]

Perhatikan ada **note**: PPN 12% hanya untuk kategori NON_SUBSIDI. LPG subsidi tidak kena pajak.

[Point ke hasil akhir]

Return OrderModel ke OrderPage. Tampilkan pesan sukses dan redirect ke detail pesanan.

Diagram ini menunjukkan **business logic** perhitungan dan **data persistence** yang structured."

SLIDE 27: SD-04 - UPDATE STATUS (SEQUENCE DIAGRAM) (3 menit)

Narasi:

"Sequence Diagram ketiga adalah **SD-04 Update Status Pesanan** - yang mengimplementasikan **auto-sync stok**.

[Point ke lifelines]

Ada **7 participants**:

- **Admin, OrderDetailPage, OrderService**
- **orders, order_items, timeline_tracks, pangkalan_stocks** (databases)

[Trace flow menampilkan detail]

Admin buka detail pesanan. OrderDetailPage memanggil `getOrderDetail(orderId)`.

OrderService melakukan **JOIN query** untuk ambil semua data terkait pesanan:

```
SELECT * FROM orders JOIN items, timeline, payment
```

code ini untuk mengambil semua data terkait pesanan

Return orderDetail, ini akan di tampilkan di halaman detail pesanan

[Trace flow update status]

Admin pilih status baru dan klik Update Status.

OrderDetailPage memanggil `updateStatus(orderId, newStatus, driverId?)`.

[Point ke validasi transisi]

OrderService query current_status, lalu `validateTransition(current, new)`.

[Point ke ALT fragment - transisi]

Ada **ALT fragment**:

- Jika **transisi tidak valid** → throw `BadRequestException` → Tampilkan error
- Jika **transisi valid** → lanjut proses diatas itu adalah validasi transisi status

[Point ke OPT fragment - SIAP_KIRIM]

Ada **OPT fragment** untuk status SIAP_KIRIM: Jika ada driverId, UPDATE orders SET driver_id.

yg diatas itu adalah update status pesanan

[Point ke update dan timeline]

Kemudian:

- **UPDATE orders** → untuk update status pesanan
- **INSERT timeline_tracks** → untuk menambahkan log timeline

[Point ke OPT fragment - SELESAI - HIGHLIGHT PENTING]

Ini bagian **paling penting!** Ada **OPT fragment** untuk status SELESAI:

OrderService:

1. **SELECT order_items** → untuk mengambil semua item dalam pesanan
2. **LOOP untuk setiap item:**

```
UPDATE pangkalan_stocks SET qty = qty + ?
```

Ini adalah **auto-sync** → stok pangkalan otomatis bertambah!

[Point ke note - Auto-sync]

Perhatikan ada **note**: ini adalah auto-sync stok pangkalan saat pesanan SELESAI.

[Point ke hasil akhir]

Return `updatedOrder`, tampilkan pesan sukses, refresh halaman.

[Kesimpulan]

Diagram ini menunjukkan **conditional logic** yang complex dan **automatic stock synchronization** yang menjadi fitur andalan SIM4LON."

SLIDE 28: SD-Voice - AI-POWERED SPEECH-TO-ORDER (3.5 menit)

Narasi:

"Sequence Diagram ini adalah **SD-Voice AI-Powered Speech-to-Order** - yang menggambarkan **technical implementation** fitur Voice Order dengan Gemini AI.

[Point ke lifelines - BANYAK participants]

Ini adalah sequence diagram paling complex. Ada **8 participants**:

- **Admin** (aktor)
- **FloatingVoiceWidget** (boundary) - UI widget untuk voice
- **useVoiceOrder** (control) - React hook
- **useSpeechRecognition** (control) - hook untuk speech recognition
- **WebSpeechAPI** (boundary) - browser API untuk speech
- **Backend API** (entity) - server NestJS
- **Gemini AI** (database) - Google AI service
- **Database** (database) - PostgreSQL

[Trace bagian 1 - Aktivasi]

Proses dimulai saat Admin klik tombol FAB microphone. Widget memanggil `startListening()`, yang kemudian meminta **izin akses microphone** dari browser.

[Trace bagian 2 - Speech Recognition]

Setelah izin diberikan, sistem masuk mode **listening** dengan UI pulsing circles.

Admin berbicara: *'Lima puluh tabung 12 kilo ke Reon'*

Ada **LOOP fragment** untuk real-time transcript - setiap audio chunk diproses dan ditampilkan secara live.

[Trace bagian 3 - Auto-Stop dan AI Parsing]

Setelah **4 detik silence**, sistem auto-stop dan status berubah ke **parsing** (ini adalah suatu proses AI yang menginterpretasikan suatu suara menjadi beberapa teks)

Sistem mengirim transkrip ke Backend API: `POST /gemini/parse-order`.

Backend memanggil **Gemini AI** yang melakukan **NLP Processing**, NLP Processing ini proses dimana AI bisa memahami bahasa manusia - misalnya kalimat seperti 'lima puluh tabung 12 kilo' menjadi data terstruktur yang bisa diproses sistem. bedanya dengan parsing yaitu kalo parsing mengekstrak data penting seperti quantity, jenis LPG, dan nama pangkalan, misalnya:

- Extract quantity: 50
- Extract product: 12kg
- Match pangkalan: 'Reon'

Return **ParsedOrderData** dengan confidence 92%.

[Trace bagian 4 - Fuzzy Matching dan Validation]

Backend melakukan **fuzzy matching** menggunakan **Levenshtein distance** - 'reon' → 'Reon' dengan 93% match. Levenshtein distance adalah metode yang digunakan untuk menghitung perbedaan antara dua string.

Kemudian validasi: pangkalan aktif? Stok cukup? Quantity valid?

[Trace bagian 5 - Konfirmasi User]

Sistem menampilkan **confirmation card** dengan hasil parsing. User bisa Cancel atau Confirm.

[Trace bagian 6 - Create Order]

Jika user confirm, sistem create order seperti biasa - INSERT orders, order_items, timeline_track.

[Trace bagian 7 - Success]

Tampilkan pesan sukses dengan order code. User bisa klik untuk lihat pesanan baru.

[Point ke note - Technology Stack]

Perhatikan ada **note** yang menjelaskan technology stack:

- Google Gemini API
- Levenshtein Fuzzy Matching
- Web Speech API (Chrome/Edge)

Diagram ini menunjukkan **integrasi modern AI** dengan **real-time processing**."

SLIDE 29: SD-07 - RECORD PAYMENT (2 menit)

Narasi:

"Sequence Diagram ini adalah **SD-07 Record Payment** - proses pencatatan pembayaran.

[Point ke lifelines]

Ada **6 participants**:

- Admin, PaymentPage, PaymentService
- order_payment_details, payment_records, orders, timeline_tracks (databases)

[Trace flow menampilkan form]

Admin buka form pembayaran. PaymentService query order_payment_details untuk ambil data pembayaran yang sudah ada - total tagihan, sudah dibayar berapa, dan sisa tagihan.

Tampilkan form dengan informasi sisa tagihan.

[Trace flow input pembayaran]

Admin pilih metode (TUNAI/TRANSFER), input jumlah, upload bukti jika transfer, lalu klik Simpan.

PaymentService melakukan:

1. `validateInput()` - validasi data input
2. Query current paid amount - total pembayaran yang sudah ada
3. `calculateRemaining()` - hitung sisa tagihan yang harus dibayar
4. `checkPaymentStatus()` - cek status pembayaran

[Point ke ALT fragment - Lunas]

Ada **ALT fragment**:

- Jika jumlah $\geq$ sisa tagihan $\rightarrow$ `setStatus(PAID, is_paid=true)`
- Jika jumlah $<$ sisa tagihan $\rightarrow$ `setStatus(PARTIAL, is_dp=true)`

[Point ke UPSERT dan INSERT]

Sistem melakukan:

- **UPSERT order_payment_details** - update atau insert
- **INSERT payment_records** - record pembayaran baru

[Point ke OPT fragment - Auto-update status]

Ada **OPT fragment** - jika pembayaran **LUNAS** DAN status = **MENUNGGU PEMBAYARAN**:

- UPDATE orders SET current_status = **DIPROSES**
- INSERT timeline_tracks dengan deskripsi 'Pembayaran diterima'

Perhatikan ada **note**: Auto-update status jika pembayaran lunas.

Diagram ini menunjukkan **fleksibilitas pembayaran** (DP/lunas) dan **automasi status**."

SLIDE 30: SD-12 - RECORD SALE (PANGKALAN) (2.5 menit)

Narasi:

"Sequence Diagram terakhir adalah **SD-12 Record Sale** - proses pencatatan penjualan oleh **Pangkalan** ke konsumen akhir.

[Point ke lifelines]

Ada **7 participants**:

- **Pangkalan** (aktor) - bukan Admin!
- **PenjualanPage, ConsumerOrderService**
- consumers, lpg_prices, pangkalan_stocks, consumer_orders, pangkalan_stock_movements (databases)

[Trace flow menampilkan form]

Pangkalan buka halaman Penjualan. ConsumerOrderService query:

- **getConsumers(pangkalanId)** - daftar konsumen milik pangkalan ini
- **getStockAndPrices(pangkalanId)** - stok dan harga LPG milik pangkalan ini

[Point ke note - Multi-tenant]

pada proses ini pangkalan bisa multi-tenant, pangkalan_id dari JWT memastikan isolasi data. Pangkalan A tidak bisa mengakses data pangkalan B.

[Trace flow input penjualan]

Pangkalan pilih konsumen, pilih jenis LPG, input quantity, lalu klik Simpan.

[Point ke validasi stok]

ConsumerOrderService query pangkalan_stocks untuk cek available qty.

[Point ke ALT fragment - Stok]

Ada **ALT fragment**:

- Jika **stok tidak mencukupi** → throw BadRequestException → Tampilkan error
- Jika **stok mencukupi** → lanjut proses

[Trace flow create sale]

Untuk stok mencukupi, sistem:

1. Generate code baru (PORD-XXXX)
2. `calculateTotal(qty, sellingPrice)`
3. `calculateProfit(selling - cost)` - hitung keuntungan
4. **INSERT consumer_orders**

[Point ke auto-deduct stok - HIGHLIGHT]

Ini yang penting:

- **UPDATE pangkalan_stocks SET qty = qty - ?** - stok **otomatis berkurang!**
- **INSERT pangkalan_stock_movements** dengan type='KELUAR', source='SALE'

Ini adalah **kebalikan dari auto-sync** - auto-sync menambah stok saat terima dari agen, auto-deduct mengurangi stok saat jual ke konsumen.

[Kesimpulan]

Diagram ini menunjukkan **complete stock lifecycle** dari pangkalan - terima dari agen (auto-sync), jual ke konsumen (auto-deduct). Semua otomatis, real-time, dan terisolasi per pangkalan."

TIMING SLIDE 21-30:

Slide	Diagram	Durasi	Kumulatif
21	AD-14 Assign Driver	1.5m	1:30

Slide	Diagram	Durasi	Kumulatif
22	AD-16 Cetak Dokumen	2m	3:30
23	AD-07 Catat Penjualan	2.5m	6:00
24	AD-17 Voice Order AI	3m	9:00
25	SD-01 Login	2.5m	11:30
26	SD-03 Create Order	2.5m	14:00
27	SD-04 Update Status	3m	17:00
28	SD-Voice AI Order	3.5m	20:30
29	SD-07 Record Payment	2m	22:30
30	SD-12 Record Sale	2.5m	25:00

Total Slide 21-30: ~25 menit

Running Total (1-30): ~33 menit ☒